DOCUMENTACION DE SISTEMA

RootCause Bitcoin Defense

18 · Guía para un desarrollador nuevo

Version analizada 0.3.0 · commit acdf4dd · generado el 2026-08-27
Documento 18 de la serie · fuente: docs/system-documentation/18-new-developer-guide.md

Contenido

1 · Los primeros treinta minutos
2 · Qué leer, y en qué orden
3 · Los cinco archivos que hay que leer sí o sí
4 · Cómo está organizado el repositorio
5 · Sigue un flujo completo, de punta a punta
6 · Dónde añadir cosas
7 · Cómo escribir pruebas aquí
8 · Convenciones que hay que respetar
9 · Partes que requieren cuidado especial
10 · Antes de cada commit
11 · Tareas iniciales apropiadas para empezar
12 · Preguntas que te harás, respondidas
13 · Itinerario de una semana

Un itinerario para entender este repositorio de verdad, en orden, con lo que hay que leer y lo que hay que ejecutar en cada paso.

> Está escrito desde la posición de quien acaba de llegar. Si algo aquí te parece obvio, es que ya no eres el destinatario.

---

1 · Los primeros treinta minutos

Paso	Acción	Por qué
1	Lee `01-system-overview.md`, incluida la sección para no técnicos	Sin entender el problema, el código parece arbitrario
2	Arranca: `node src/server.js`	Verlo funcionando cambia cómo lees el código
3	Abre `http://127.0.0.1:8787` y pulsa **Ejecutar análisis**	Verás hallazgos reales sobre el inventario de demostración
4	Ve a **Laboratorio** y pulsa **Simular gasto no autorizado**	Observa cómo un evento se convierte en incidente
5	`curl http://127.0.0.1:8787/api/posture`	La aplicación auditándose a sí misma
6	Prueba a enviar un secreto	Ver la sección 2

1.1 · Comprueba tú mismo el invariante central

Esta petición **debe** ser rechazada:

```
curl -i -X POST http://127.0.0.1:8787/api/wallets \
 -H "content-type: application/json" \
 -H "x-rootcause-request: 1" \
 -d '{"name":"prueba","seed":"palabras que nunca deberias escribir"}'
```

Respuesta esperada: `422` con `SECRET_MATERIAL_REJECTED`.

Si algún día esa petición devuelve otra cosa, el producto ha dejado de ser lo que dice ser. Es literalmente lo que verifica `check-security-claims.js` en cada build.

---

2 · Qué leer, y en qué orden

2.1 · Primer día — entender el porqué

#	Documento	Qué te da
1	`../MANUAL_PARA_NOVATOS.md`	El problema, sin dar nada por sabido
2	`../INCIDENT-COLDCARD-2026.md`	El incidente que originó el producto
3	`../LIMITACIONES.md`	**Lo que el producto no puede hacer.** Léelo pronto: evita proponer cosas imposibles
4	`16-glossary.md`	Vocabulario, para no perderte

2.2 · Segundo día — entender la forma

#	Documento	Qué te da
5	`03-architecture.md`	Las capas y por qué las dependencias apuntan hacia dentro
6	`04-code-map.md`	Qué archivo hace qué
7	`07-database.md`	El modelo de datos completo

2.3 · Tercer día — entender el detalle

#	Documento	Qué te da
8	`06-deep-code-explanation.md`	Cómo funciona por dentro
9	`../reglas/README.md`	Las 26 reglas, una a una
10	`05-technical-reference.md`	Referencia de consulta; **no lo leas de corrido**

---

3 · Los cinco archivos que hay que leer sí o sí

Si solo tienes tiempo para cinco, que sean estos, en este orden:

#	Archivo	Líneas	Por qué
1	`src/domain/secret-guard.js`	100	**Define el producto.** Todo lo demás se apoya en que esto funcione
2	`src/config.js`	80	Gobierna todo el comportamiento del proceso
3	`src/server.js`	153	El *composition root*: aquí ves de qué está hecho el sistema
4	`src/services/defense-service.js`	541	El orquestador; toda escritura pasa por aquí
5	`src/domain/rule-engine.js`	622	El motor. Empieza por `evaluateState` al final y sube

Después, `src/app.js` para entender la frontera HTTP.

---

4 · Cómo está organizado el repositorio

```
src/domain/          Las reglas. No sabe de HTTP ni de archivos
src/services/        Los casos de uso. Aquí se escribe estado
src/infrastructure/  Cifrado, auditoria y RPC
src/api/ src/app.js  La frontera HTTP
src/web/static/      El panel, sin framework y sin build
config/              Avisos, politica y el catalogo de reglas
scripts/             Las puertas de calidad y las utilidades
test/                64 pruebas con node:test
docs/                La documentacion, gobernada por un validador de estilo
packaging/windows/   Empaquetador, instalador y lanzadores
```

La regla que explica el resto: el dominio no importa nada de fuera. Si te ves añadiendo un `import` de `infrastructure` dentro de `domain`, estás resolviendo el problema en el sitio equivocado.

---

5 · Sigue un flujo completo, de punta a punta

El ejercicio que más rápido enseña el sistema. Sigue el registro de una wallet:

#	Archivo	Qué mirar
1	`src/web/static/app.js` → `submitWallet`	Cómo el formulario compone la carga útil
2	`src/web/static/app.js` → `api`	Dónde se añade `x-rootcause-request`
3	`src/app.js` → `validateMutationRequest`	Por qué sin esa cabecera es `403`
4	`src/app.js` → `readJson`	Cómo se aplica el límite **durante** la lectura
5	`src/api/router.js`	La línea que despacha a `addWallet`
6	`src/services/defense-service.js` → `addWallet`	La secuencia completa
7	`src/domain/secret-guard.js` → `assertNoSecretMaterial`	El rechazo
8	`src/services/defense-service.js` → `normalizeWallet`	Validación y valores derivados

#	Archivo	Qué mirar
9	`src/domain/rule-engine.js` → `evaluateWallet`	Dónde nacen los hallazgos
10	`src/domain/firmware.js` → `advisoryAffectsWallet`	La comparación que decide
11	`src/services/defense-service.js` → `mergeIncidents`	Cómo un hallazgo se vuelve incidente
12	`src/infrastructure/audit-log.js` → `appendAuditEntry`	Cómo se encadena la traza
13	`src/infrastructure/encrypted-store.js` → `save`	Cómo acaba cifrado en disco

Trece paradas. Es todo el sistema.

---

6 · Dónde añadir cosas

Quieres añadir	Dónde va	Qué no olvidar
Una regla nueva	`src/domain/rule-engine.js` o `posture.js`	**Su ficha en `config/rules-catalog.json`.** Sin ella el build cae
Un aviso de fabricante	`config/advisories.json`	`verifiedAt`, y que `vendor` y `models` coincidan con lo que escribe el usuario
Un endpoint	`src/api/router.js` + método en `DefenseService`	Que las mutaciones exijan la cabecera; una prueba en `test/api.test.js`
Un campo del modelo	`normalizeWallet` o `normalizeSigner`	Su validador, su valor por defecto y la fila en `07-database.md`
Un método RPC	`READ_ONLY_METHODS`	[FALLO] **Solo de lectura.** Tres controles independientes lo comprueban
Una comprobación de calidad	`scripts/`	Que **declare su cobertura** y salga con código ≠ 0 al fallar
Un documento	`docs/`	Enlazarlo desde `docs/INDEX.md`, o el build cae
Algo en el panel	`src/web/static/app.js`	**`escapeHtml` en toda interpolación.** Sin excepciones

6.1 · Añadir una regla, paso a paso

```
# 1. Implementarla en src/domain/rule-engine.js con makeFinding(...)
# 2. Añadir su ficha completa a config/rules-catalog.json
node scripts/generate-rule-docs.js
node --test
node scripts/check-doc-style.js
```

`generate-rule-docs.js` crea la ficha en `docs/reglas/` y actualiza la página publicada. **No edites esos archivos a mano**: la siguiente ejecución los pisa y el `--check` de CI habría fallado antes.

---

7 · Cómo escribir pruebas aquí

Se usa `node:test` y `node:assert/strict`. Sin framework, sin `describe`, sin mocks de librería.

```
import assert from "node:assert/strict";
import test from "node:test";
import { evaluateWallet } from "../src/domain/rule-engine.js";

test("describe el comportamiento, no la implementacion", () => {
 const findings = evaluateWallet(wallet, { advisories: [], highValueThresholdSats: 1000 });
 assert.equal(findings.length, 1);
 assert.equal(findings[0].code, "BTC-CUSTODY-001");
});
```

La convención más importante del repositorio: cada prueba de detección lleva su **par negativo**. No basta con comprobar que la regla dispara; hay que comprobar que **no** dispara cuando no debe. Mira `test/attack-patterns.test.js`: barrido frente a pago parcial, ráfaga frente a gastos espaciados, filtración de tu fabricante frente a filtración ajena.

Para probar con servidor real, copia el patrón de `test/api.test.js`: un `MemoryStore`, un `DefenseService` real y `server.listen(0)`.

---

8 · Convenciones que hay que respetar

Convención	Detalle
Módulos ESM	`import`, nunca `require`
Solo `node:` y rutas relativas	Cualquier otro especificador tumba `check-local-only.js`
Sin dependencias	Ni de producción, ni de desarrollo, ni de herramientas
Indentación	2 espacios, LF, sin espacios al final
Nombres	`camelCase`; clases en `PascalCase`
Errores	Siempre con `statusCode`; `code` cuando el cliente deba distinguirlo
Comentarios	**Explican el porqué, no el qué.** En español, en prosa, con separadores de sección donde ayude
Concatenación de cadenas	El repositorio usa + en vez de plantillas en muchos sitios. **Mantén el estilo del archivo que tocas**
Documentación	Un H1, máximo H4, tablas con cabecera, bloques de código con lenguaje
Versión	Está en **dos** sitios: `package.json` y `src/api/router.js`

Un comentario que dice *«incrementa el contador»* sobre `count += 1` es ruido. Uno que dice *por qué* existe ese contador es lo que aquí se espera.

---

9 · Partes que requieren cuidado especial

Zona	Por qué	Qué hacer antes de tocarla
`secret-guard.js`	**Define el producto**	Ejecuta `check-security-claims.js` antes y después
`READ_ONLY_METHODS`	Un método de escritura convierte esto en algo que puede perder fondos	Entiende los tres controles que lo vigilan
La CSP de `src/app.js`	Un script la verifica en cada build	Lee qué directivas son obligatorias
`findingId` y `mergeIncidents`	Cambiar el hash resolvería en falso todo el historial	Entiende el ciclo de vida completo
`canonicalize` en `audit-log.js`	Cambiarlo invalida todas las cadenas ya escritas	No lo toques sin una razón de peso
`advisoryAffectsWallet`	Un falso negativo oculta fondos comprometidos; un falso positivo provoca una migración innecesaria	Añade pruebas en los dos sentidos
`loadConfig`	Un valor por defecto distinto puede abrir el producto	Ejecuta `check-security-claims.js`
Los valores por defecto de `normalizeSigner`	Determinan qué reglas disparan	Piensa qué pasa si el usuario omite el campo
Cualquier `innerHTML` del panel	Sin `escapeHtml` es XSS	Escapa siempre

---

10 · Antes de cada commit

```
node --test
node scripts/validate-repo.js
node scripts/check-local-only.js
node scripts/check-docs.js
node scripts/check-doc-style.js
node scripts/generate-rule-docs.js --check
node scripts/check-security-claims.js
```

O todo de una vez:

```
pnpm check
```

Antes de un push, además:

```
node scripts/check-history-secrets.js --staged
node scripts/sync-plugin.js --check
```

---

11 · Tareas iniciales apropiadas para empezar

Ordenadas de menor a mayor exposición al núcleo del sistema. Todas salen de `15-risks-and-technical-debt.md`, así que son trabajo real, no ejercicios.

#	Tarea	Dificultad	Qué te enseña
1	Añadir un comentario a `.env.example` aclarando que el sistema no lee `.env`	Trivial	El flujo de contribución completo

#	Tarea	Dificultad	Qué te enseña
2	Registrar el `x-request-id` en el log de los 5xx	Muy baja	La capa HTTP
3	Escribir las pruebas de `riskScore` y `riskLevel` en sus fronteras	Baja	Cómo se prueba aquí
4	Avisar en el log cuando un valor de configuración se recorta	Baja	`loadConfig` y sus caídas silenciosas
5	Construir un `Set` de hashes de aprobación en vez de la búsqueda cuadrática	Baja	El motor de correlaciones
6	Probar la concurrencia de `mutate()`: dos altas simultáneas deben producir dos wallets	Media	La cola de escritura, y por qué hace falta en un runtime monohilo
7	Probar el límite de tasa y el límite de cuerpo	Media	Pruebas de integración HTTP
8	Probar `snapshot()` cuando el nodo falla a mitad de las cuatro llamadas	Media	El cliente RPC y su política de no lanzar
9	Hacer efectiva la caducidad de `expiresAt`	Media	El motor de reglas y su modelo de datos
10	Añadir una regla nueva, con su ficha	Media-alta	El ciclo completo: motor, catálogo, generador, prueba

Empieza por la 1 o la 2 para recorrer el flujo entero —editar, validar, comprometer— sin arriesgar nada. La 6 es la que más valor aporta al proyecto ahora mismo.

> Si buscas un modelo de cómo se prueba aquí, lee `test/incident-lifecycle.test.js`: cubre el ciclo de vida completo de un incidente y es la prueba más reciente del repositorio.

---

12 · Preguntas que te harás, respondidas

Pregunta	Respuesta
¿Por qué no hay TypeScript?	`../JAVASCRIPT_PARA_ROOTCAUSE.md`. Resumen: sería una cadena de compilación y un árbol de dependencias
¿Por qué no hay ESLint?	Sería una dependencia. El hueco es real y está registrado
¿Puedo añadir una librería pequeña?	**No.** `check-local-only.js` tumba el build, y con razón: es el claim central de distribución
¿Por qué el panel no usa un framework?	Mismo motivo. Y funciona: 601 líneas para cinco vistas
¿Por qué hay dos copias de `src/`?	`build/portable/` es el paquete ensamblado, está en `.gitignore` y se regenera
¿Por qué no puedo editar `docs/reglas/`?	Se genera desde `config/rules-catalog.json`
¿Por qué no puedo editar `plugin/`?	Se genera desde `.claude/`
¿Por qué la versión está en dos sitios?	`package.json` y `src/api/router.js`. Un validador comprueba que coincidan
¿Dónde están los logs en Windows?	En la ventana de consola del lanzador. **Se pierden al cerrarla**
¿Puedo borrar una wallet mal registrada?	Hoy no. No hay endpoint de borrado, a propósito

13 · Itinerario de una semana

Día	Objetivo	Entregable
1	Entender el problema y ver el sistema funcionando	Sabes explicar por qué actualizar el firmware no repara una semilla
2	Entender la arquitectura y el mapa de código	Sabes en qué capa iría un cambio dado
3	Seguir el flujo de las 13 paradas de la sección 5	Sabes qué pasa entre el formulario y el disco
4	Leer las 26 fichas de regla y ejecutar la auditoría de 14 etapas	Sabes qué detecta el producto y qué no
5	Hacer la tarea 1 o 2 de la sección 11, con todas las puertas en verde	Tu primer commit

Siguiente	
Matriz de trazabilidad	`19-traceability-matrix.md`
Volver al índice	`README.md`